

LEGO Education SPIKE Prime

Python — ściągą: silniki, poruszanie i czujniki (firmware SPIKE 3)

Jak zbudowany jest program

Każdy komponent ma swój moduł, który najpierw trzeba zaimportować. Porty (A–F) podaje się przez stałe z modułu port. Polecenia ruchu są „awaitable” — domyślnie startują w tle i program leci dalej; żeby poczekać na zakończenie, używa się await wewnątrz funkcji async uruchomionej przez runloop.run().

```
import motor
import motor_pair
import color_sensor, distance_sensor, force_sensor
import runloop
from hub import port, motion_sensor

async def main():
    await motor.run_for_degrees(port.A, 360, 720)

runloop.run(main())
```

Wartości to liczby całkowite (int). Zamiast ułamków używa się mniejszych jednostek — np. 500 ms zamiast 0,5 s.

Porty i status ruchu

port.A ... port.F — gniazda na hubie (wartości 0–5).

Funkcje ruchu po await zwracają status: motor.READY, RUNNING, STALLED (zablokowany), CANCELED, ERROR, DISCONNECTED.

Silniki — moduł motor

Sterowanie pojedynczym silnikiem podłączonym do jednego portu. import motor

Ruch — uruchamianie i obracanie

Funkcja	Działanie / zwraca
run(port, velocity)	Kręci silnikiem ze stałą prędkością aż do kolejnej komendy (nie czeka).
run_for_degrees(port, degrees, velocity)	Obraca o zadaną liczbę stopni. Awaitable — zwraca status.
run_for_time(port, duration, velocity)	Kręci przez podany czas (ms). Awaitable.
run_to_relative_position(port, position, velocity)	Obraca do pozycji względem aktualnej. Awaitable.
run_to_absolute_position(port, position, velocity)	Obraca do bezwzględnej pozycji (kąt 0–359°). Awaitable.
set_duty_cycle(port, pwm)	Uruchamia z konkretnym PWM (-10000...10000), bez regulacji prędkości.
stop(port)	Zatrzymuje silnik (domyślnie hamowanie).

Odczyt stanu silnika

Funkcja	Działanie / zwraca
<code>velocity(port)</code>	Aktualna prędkość w °/s.
<code>absolute_position(port)</code>	Pozycja bezwzględna 0–359°.
<code>relative_position(port)</code>	Pozycja względna (skumulowane stopnie).
<code>reset_relative_position(port, position)</code>	Ustawia punkt odniesienia dla pozycji względnej.
<code>get_duty_cycle(port)</code>	Aktualny PWM silnika.

Najważniejsze parametry

Parametr	Typ / zakres	Opis
velocity	int, °/s	Prędkość. Zakres zależy od typu silnika: mały ±660, średni ±1110, duży ±1050. Znak = kierunek.
degrees	int, °	Liczba stopni do obrócenia (<code>run_for_degrees</code>).
duration	int, ms	Czas pracy w milisekundach (<code>run_for_time</code>).
position	int, °	Docelowy kąt (0–359 dla <code>absolute</code>).
acceleration	int, 1–10000	Przyspieszenie w °/s². Argument opcjonalny (domyślnie 1000).
deceleration	int, 1–10000	Hamowanie w °/s². Opcjonalny (domyślnie 1000).
stop	stała	Zachowanie po zatrzymaniu — patrz tabela niżej.
direction	stała	Kierunek dla <code>absolute</code> : <code>CLOCKWISE</code> , <code>COUNTERCLOCKWISE</code> , <code>SHORTEST_PATH</code> , <code>LONGEST_PATH</code> .

Zachowanie po zatrzymaniu (parametr `stop`)

Parametr	Typ / zakres	Opis
<code>motor.COAST</code>	0	Swobodny wybieg do zatrzymania.
<code>motor.BRAKE</code>	1	Hamuje i utrzymuje hamowanie (domyślne).
<code>motor.HOLD</code>	2	Aktywnie trzyma pozycję.
<code>motor.CONTINUE</code>	3	Kręci dalej z bieżącą prędkością do kolejnej komendy.
<code>motor.SMART_COAST</code>	4	Hamuje, potem wybieg; kompensuje błędy przy następnej komendzie.
<code>motor.SMART_BRAKE</code>	5	Hamuje i utrzymuje; kompensuje niedokładności.

```
# Obrót o 360° z prędkością 720 °/s i czekaniem na koniec
await motor.run_for_degrees(port.A, 360, 720)
```

```
# Start ciągły, po 2 s stop
motor.run(port.A, 1000)
await runloop.sleep_ms(2000)
motor.stop(port.A, stop=motor.HOLD)
```

Poruszanie się — moduł `motor_pair`

Synchronizuje dwa silniki jako napęd pojazdu (`drivebase`): startują i zatrzymują się razem. `import motor_pair`

Najpierw trzeba sparować silniki w „slocie” pary, a potem używać tego slotu we wszystkich komendach.

```
motor_pair.pair(motor_pair.PAIR_1, port.A, port.B) # lewy, prawy
```

Konfiguracja pary

Funkcja	Działanie / zwraca
<code>pair(pair, left_motor, right_motor)</code>	Łączy dwa silniki w parę (slot).
<code>unpair(pair)</code>	Rozłącza parę.
<code>stop(pair)</code>	Zatrzymuje obie silniki pary.

Sloty: `motor_pair.PAIR_1` (0), `PAIR_2` (1), `PAIR_3` (2).

Jazda ze sterowaniem (steering)

Model „steering”: jeden parametr skrótu od -100 (ostro w lewo) przez 0 (prosto) do 100 (ostro w prawo).

Funkcja		Działanie / zwraca
move(pair, steering)		Jedzie ze stałą prędkością aż do kolejnej komendy (nie czeka).
move_for_degrees(pair, degrees, steering)		Jedzie przez zadaną liczbę stopni. Awaitable.
move_for_time(pair, duration, steering)		Jedzie przez podany czas (ms). Awaitable.
Parametr	Typ / zakres	Opis
steering	int, -100...100	Kierunek/skręt: -100 lewo, 0 prosto, 100 prawo.
velocity	int, °/s	Prędkość (opcjonalna, domyślnie 360). Zakres jak dla silnika.
degrees / duration	int	Dystans w stopniach lub czas w ms.
acceleration / deceleration	int, 1–10000	Przyspieszenie / hamowanie (opcjonalne).
stop	stała	Zachowanie po stopie (jak w module motor).

Jazda „czołgowa” (tank) — osobno lewy i prawy

Model „tank”: podajesz prędkość każdego koła osobno. Przeciwnie znaki = obrót w miejscu.

Funkcja		Działanie / zwraca
move_tank(pair, left_velocity, right_velocity)		Jazda na dwóch prędkościach do kolejnej komendy.
move_tank_for_degrees(pair, degrees, left_v, right_v)		Tank przez zadane stopnie. Awaitable.
move_tank_for_time(pair, left_v, right_v, duration)		Tank przez podany czas (ms). Awaitable.
Parametr	Typ / zakres	Opis
left_velocity	int, °/s	Prędkość lewego silnika.
right_velocity	int, °/s	Prędkość prawego silnika.
degrees / duration	int	Dystans (stopnie) lub czas (ms).

```
motor_pair.pair(motor_pair.PAIR_1, port.A, port.B)
```

```
await motor_pair.move_for_degrees(motor_pair.PAIR_1, 360, 0) # prosto
await motor_pair.move_tank_for_time(motor_pair.PAIR_1, 1000, -1000, 2000) # obrót w miejscu
```

Czujniki

Czujnik koloru — color_sensor

Reaguje na kolory i natężenie odbitego światła. `import color_sensor`. Rozpoznaje: czerwony, zielony, niebieski, magenta, żółty, pomarańczowy, lazurowy, czarny, biały.

Funkcja	Działanie / zwraca
<code>color(port)</code>	Zwraca wykryty kolor (int) — porównuj ze stałymi z modułu <code>color</code> , np. <code>color.RED</code> .
<code>reflection(port)</code>	Natężenie odbitego światła 0–100%.
<code>rgb(port)</code>	Krotka (red, green, blue, intensity) — składowe i ogólna jasność.

```
import color, color_sensor
if color_sensor.color(port.A) == color.RED:
    print("Czerwony!")
```

Stałe kolorów (`color`): BLACK 0, MAGENTA 1, PURPLE 2, BLUE 3, AZURE 4, TURQUOISE 5, GREEN 6, YELLOW 7, ORANGE 8, RED 9, WHITE 10, UNKNOWN -1.

Czujnik odległości — distance_sensor

Mierzy odległość ultradźwiękowo i steruje swoimi 4 diodami. `import distance_sensor`

Funkcja	Działanie / zwraca	
<code>distance(port)</code>	Odległość w milimetrach. Zwraca -1, gdy brak poprawnego odczytu.	
<code>get_pixel(port, x, y)</code>	Jasność wybranej diody (x,y w zakresie 0–3).	
<code>set_pixel(port, x, y, intensity)</code>	Ustawia jasność wybranej diody.	
<code>show(port, pixels)</code>	Ustawia wszystkie 4 diody naraz (lista wartości).	
<code>clear(port)</code>	Gasi wszystkie diody czujnika.	
Parametr	Typ / zakres	Opis
distance (zwracane)	int, mm	Zmierzona odległość; -1 = brak odczytu.
x, y	int, 0–3	Współrzędne diody na czujniku.
intensity	int	Jasność diody (jak mocno ma świecić).

Czujnik siły — force_sensor

Mierzy nacisk i działa jak przycisk. `import force_sensor`

Funkcja	Działanie / zwraca
<code>force(port)</code>	Zmierzona siła w decyniutonach (dN), zakres 0–100 (maks. 10 N).
<code>pressed(port)</code>	True, jeśli przycisk czujnika jest wciśnięty.
<code>raw(port)</code>	Surowa, niekalibrowana wartość siły.

```
import force_sensor
while True:
    motor.run(port.A, force_sensor.force(port.B) * 5)
```

Czujnik ruchu (w hubie) — motion_sensor

Wbudowany w hub żyroskop/akcelerometr: orientacja, gesty, kąty. `from hub import motion_sensor`

Funkcja	Działanie / zwraca
<code>tilt_angles()</code>	Krotka (yaw, pitch, roll) w decystopniach.
<code>acceleration(raw_unfiltered)</code>	Przyspieszenie (x, y, z) w mili-g (1/1000 G).
<code>angular_velocity(raw_unfiltered)</code>	Prędkość kątowna (x, y, z) w decystopniach/s.
<code>gesture()</code>	Wykryty gest: TAPPED, DOUBLE_TAPPED, SHAKEN, FALLING, UNKNOWN.
<code>up_face()</code>	Która ściana huba jest skierowana do góry.
<code>stable()</code>	True, gdy hub leży nieruchomo.
<code>tap_count()</code>	Liczba stuknięć od startu / od resetu.
<code>reset_tap_count()</code>	Zeruje licznik stuknięć.
<code>quaternion()</code>	Orientacja huba jako kwaternion (w, x, y, z).
<code>reset_yaw(angle)</code>	Ustawia nowy punkt odniesienia dla kąta yaw.
<code>get_yaw_face()</code> / <code>set_yaw_face(up)</code>	Odczyt / ustawienie ściany odniesienia dla yaw.

Ściany huba: TOP, FRONT, RIGHT, BOTTOM, BACK, LEFT. Gesty i ściany podaje się jako stałe `motion_sensor`.*

Czekanie i reagowanie — runloop

Funkcja	Działanie / zwraca
<code>runloop.run(*functions)</code>	Uruchamia jedną lub wiele funkcji async równolegle.
<code>runloop.sleep_ms(duration)</code>	Pauza nieblokująca (await) — inne zadania mogą działać dalej.
<code>runloop.until(function, timeout=0)</code>	Czeka, aż warunek (funkcja zwracająca bool) będzie True lub minie timeout.

```
def widzi_czerwony():
    return color_sensor.color(port.A) == color.RED

async def main():
    await runloop.until(widzi_czerwony)    # czekaj na czerwony
    motor_pair.stop(motor_pair.PAIR_1)

runloop.run(main())
```

Wskazówka: w „ciasnej” pętli `while` wstaw `await runloop.sleep_ms(1)`, żeby dać szansę innym zadaniom.

Na podstawie LEGO Education SPIKE Prime Python Documentation (SPIKE 3, wersja 3.4.3), tuftsceeo.github.io/SPIKEPythonDocs. LEGO® i SPIKE™ są znakami towarowymi The LEGO Group; materiał nieoficjalny.